

11.21补充板子:

线性筛, exgcd, 自动取模类, 动态组合数, 静态组合数, NTT, 线性基, 素性测试与因式分解, 圆方树, HLD, 线段树, 一般图最大匹配

1.线性筛

- 质数 + 最小质因子 + 欧拉函数 + 莫比乌斯函数 + 莫比乌斯函数前缀和 (杜教筛 $O(n^{\frac{2}{3}})$)

```
std::unordered_map<int, Z> fMu;

std::vector<int> minp, primes, phi, mu;
std::vector<i64> sphi;

void sieve(int n) {
    minp.assign(n + 1, 0);
    phi.assign(n + 1, 0);
    sphi.assign(n + 1, 0);
    mu.assign(n + 1, 0);
    primes.clear();
    phi[1] = 1;
    mu[1] = 1;

    for (int i = 2; i <= n; i++) {
        if (minp[i] == 0) {
            minp[i] = i;
            phi[i] = i - 1;
            mu[i] = -1;
            primes.push_back(i);
        }

        for (auto p : primes) {
            if (i * p > n) {
                break;
            }
            minp[i * p] = p;
            if (p == minp[i]) {
                phi[i * p] = phi[i] * p;
                break;
            }
            phi[i * p] = phi[i] * (p - 1);
            mu[i * p] = -mu[i];
        }
    }
}
```

```

    for (int i = 1; i <= n; i++) {
        sphi[i] = sphi[i - 1] + phi[i];
        mu[i] += mu[i - 1];
    }
}

Z sumMu(int n) {
    if (n <= N) {
        return mu[n];
    }
    if (fMu.count(n)) {
        return fMu[n];
    }
    if (n == 0) {
        return 0;
    }
    Z ans = 1;
    for (int l = 2, r; l <= n; l = r + 1) {
        r = n / (n / l);
        ans -= (r - l + 1) * sumMu(n / l);
    }
    return ans;
}

```

2.扩展欧几里得

- exgcd: 求出 a 和 b 的最大公约数, 同时求出满足 $ax + by = g$ 的一组整系数解 x 和 y
- mod_inv: 求解 $ax \equiv 1(mod\ p)$ 的最小正整数解
- get: 求解 $ax \equiv b(mod\ c)$ 的最小正整数解
- 在求解不定方程时, 推荐使用`get`来求, 以防溢出
- 通解为 $X = x_0 + k * \frac{b}{g}, Y = y_0 - k * \frac{a}{g}$

```

11 exgcd(11 a, 11 b, 11 &x, 11 &y) {
    if(b == 0) {
        x = 1;
        y = 0;
        return a;
    }
    11 g = exgcd(b, a % b, y, x);
    y -= a / b * x;
    return g;
}
11 mod_inv(11 a, 11 p) {
    11 x, y;
    exgcd(a, p, x, y);
    return (x % p + p) % p;
}
11 get(11 a, 11 b, 11 c) {
    b = (b % c + c) % c;
    11 g = gcd(a, c);

```

```

    if(b % g != 0) {
        return -1;
    }
    a /= g;
    b /= g;
    c /= g;
    return b * mod_inv(a, c) % c;
}

```

3.取模运算类 (Z)

```

constexpr int P = 998244353;
using ll = long long;
int norm(int x) {
    if(x < 0) {
        x += P;
    }
    if(x >= P) {
        x -= P;
    }
    return x;
}
template<class T>
T power(T a, ll b) {
    T res = 1;
    while(b) {
        if(b & 1) res *= a;
        a *= a;
        b >>= 1;
    }
    return res;
}
struct Z {
    int x;
    Z(int x = 0) : x(norm(x % P)) {}
    Z(ll x) : x(norm(x % P)) {}
    int val() const {
        return x;
    }
    Z operator-() const {
        return Z(norm(P - x));
    }
    Z inv() const {
        assert(x != 0);
        return power(*this, P - 2);
    }
    Z &operator*=(const Z &rhs) {
        x = ll(x) * rhs.x % P;
        return *this;
    }
    Z &operator+=(const Z &rhs) {
        x = norm(x + rhs.x);
    }
}

```

```

        return *this;
    }
    Z &operator--(const Z &rhs) {
        x = norm(x - rhs.x);
        return *this;
    }
    Z &operator/=(const Z &rhs) {
        return *this *= rhs.inv();
    }
    friend Z operator*(const Z &lhs, const Z &rhs) {
        Z res = lhs;
        res *= rhs;
        return res;
    }
    friend Z operator+(const Z &lhs, const Z &rhs) {
        Z res = lhs;
        res += rhs;
        return res;
    }
    friend Z operator-(const Z &lhs, const Z &rhs) {
        Z res = lhs;
        res -= rhs;
        return res;
    }
    friend Z operator/(const Z &lhs, const Z &rhs) {
        Z res = lhs;
        res /= rhs;
        return res;
    }
    friend istream &operator>>(istream &is, Z &a) {
        ll v;
        is >> v;
        a = Z(v);
        return is;
    }
    friend ostream &operator<<(ostream &os, const Z &a) {
        return os << a.val();
    }
};

```

4.动态组合数 (With Z)

```

struct Comb {
    int n;
    vector<Z> _fac;
    vector<Z> _invfac;
    vector<Z> _inv;

    Comb() : n{0}, _fac{1}, _invfac{1}, _inv{0} {}
    Comb(int n) : Comb() {
        init(n);
    }
}

```

```

void init(int m) {
    if(m <= n) return ;
    _fac.resize(m + 1);
    _invfac.resize(m + 1);
    _inv.resize(m + 1);
    for(int i = n + 1; i <= m; i++) {
        _fac[i] = _fac[i - 1] * i;
    }
    _invfac[m] = _fac[m].inv();
    for(int i = m; i > n; i--) {
        _invfac[i - 1] = _invfac[i] * i;
        _inv[i] = _invfac[i] * _fac[i - 1];
    }
    n = m;
}

Z fac(int m) {
    if(m > n) init(2 * m);
    return _fac[m];
}

Z invfac(int m) {
    if(m > n) init(2 * m);
    return _invfac[m];
}

Z inv(int m) {
    if(m > n) init(2 * m);
    return _inv[m];
}

Z binom(int n, int m) {
    if(n < m || m < 0) return 0;
    return fac(n) * invfac(m) * invfac(n - m);
}
} comb;

```

5.静态组合数

```

const int MOD = 998244353;
const int N = 5005;

void add(int &x, int y) {
    x += y - MOD; x += x >> 31 & MOD;
}

int fac[N + 1], inv_fac[N + 1];
int fast_pow(int a, int b) {
    int res = 1;
    while(b) {
        if(b & 1) res = 1LL * res * a % MOD;
        a = 1LL * a * a % MOD;
        b >>= 1;
    }
    return res;
}

int inv(int x) {

```

```

        return fast_pow(x, MOD - 2);
    }
    int C(int n, int m) {
        if(n < 0 || n < m) return 0;
        return 1LL * fac[n] * inv_fac[m] % MOD * inv_fac[n - m] % MOD;
    }

    void init() {
        fac[0] = inv_fac[0] = 1;
        for(int i = 1; i <= N; i++) {
            fac[i] = 1LL * fac[i - 1] * i % MOD;
        }
        inv_fac[N] = inv(fac[N]);
        for(int i = N - 1; i >= 1; i--) {
            inv_fac[i] = 1LL * inv_fac[i + 1] * (i + 1) % MOD;
        }
    }
}

```

6.线性基

```

class XorBasis {
    vector<ll> b;

    int num;
    ll _or;

    bool canBeZero;
    vector<ll> basis;

    vector<int> rightMost;
    int rightMostZero;
public:
    XorBasis(vector<ll> &a) {
        b.resize(64);
        rightMost.resize(b.size(), 0);
        rightMostZero = -1;
        num = 0;
        _or = 0;
        canBeZero = false;

        for(ll v : a) {
            insert(v);
        }
    };
    XorBasis() {
        b.resize(64);
        rightMost.resize(b.size(), 0);
        rightMostZero = -1;
        num = 0;
        _or = 0;
        canBeZero = false;
    }
};

```

```

}
bool insert(ll v) {
    _or |= v;
    for(int i = b.size() - 1; i >= 0; i--) {
        if(!(v >> i)) {
            continue;
        }
        if(b[i] == 0) {
            b[i] = v;
            num++;
            return true;
        }
        v ^= b[i];
    }
    canBeZero = true;
    return false;
}
bool insertRightMost(int idx, ll v) {
    for(int i = b.size() - 1; i >= 0; i--) {
        if(!(v >> i)) {
            continue;
        }
        if(b[i] == 0) {
            b[i] = v;
            rightMost[i] = idx;
            num++;
            return true;
        }
        if(idx > rightMost[i]) {
            swap(idx, rightMost[i]);
            swap(v, b[i]);
        }
        v ^= b[i];
    }
    canBeZero = true;
    rightMostZero = max(rightMostZero, idx);
    return false;
}
bool decompose(ll v) {
    for(int i = b.size() - 1; i >= 0; i--) {
        if(!(v >> i)) {
            continue;
        }
        if(b[i] == 0) {
            return false;
        }
        v ^= b[i];
    }
    return true;
}
ll maxXor() {
    ll res = 0;
    for(int i = b.size() - 1; i >= 0; i--) {

```

```

        res = max(res, res ^ b[i]);
    }
    return res;
}
11 maxXorWithVal(11 v) {
    11 res = v;
    for(int i = b.size() - 1; i >= 0; i--) {
        res = max(res, res ^ b[i]);
    }
    return res;
}
11 maxXorWithLowerIndex(int lowerIndex) {
    11 res = 0;
    for(int i = b.size() - 1; i >= 0; i--) {
        if(!(res >> i & 1) && rightMost[i] >= lowerIndex) {
            res = max(res, res ^ b[i]);
        }
    }
    return res;
}
11 minXor() {
    if(canBeZero) {
        return 0;
    }
    for(int i = 0; i < b.size(); i++) {
        if(b[i] > 0) {
            return b[i];
        }
    }
    return -1; //基为空
}
void initOnce() {
    if(!basis.empty()) {
        return;
    }
    vector<11> tmp = b;
    for(int i = 0; i < tmp.size(); i++) {
        if(tmp[i] == 0) {
            continue;
        }
        for(int j = i - 1; j >= 0; j--) {
            if(tmp[i] >> j & 1) {
                tmp[i] ^= tmp[j];
            }
        }
        basis.push_back(tmp[i]);
    }
}
11 kthXor(11 k) {
    initOnce();
    if(canBeZero) {
        k--;
    }
}

```

```

        if(k >= (1LL << basis.size())) {
            return -1;
        }
        ll _xor = 0;
        for(int i = 0; i < basis.size(); i++) {
            if(k >> i & 1) {
                _xor ^= basis[i];
            }
        }
        return _xor;
    }
    void merge(const XorBasis& other) {
        for(int i = other.b.size() - 1; i >= 0; i--) {
            ll v = other.b[i];
            if(v > 0) {
                insert(v);
            }
        }
    }
};

```

7.NTT

```

// --- 依赖之前的 Z 类 (模数必须是 998244353) ---
// ... (此处粘贴之前的 Z 类代码) ...

// --- NTT 核心部分 ---
namespace NTT {
    std::vector<int> rev;
    std::vector<Z> roots{0, 1};

    void dft(std::vector<Z> &a) {
        int n = a.size();
        if (int(rev.size()) != n) {
            int k = __builtin_ctz(n) - 1;
            rev.resize(n);
            for (int i = 0; i < n; i++)
                rev[i] = rev[i >> 1] >> 1 | (i & 1) << k;
        }
        for (int i = 0; i < n; i++)
            if (rev[i] < i) std::swap(a[i], a[rev[i]]);

        if (int(roots.size()) < n) {
            int k = __builtin_ctz(roots.size());
            roots.resize(n);
            while ((1 << k) < n) {
                Z e = power(Z(3), (P - 1) >> (k + 1));
                for (int i = 1 << (k - 1); i < (1 << k); i++) {
                    roots[2 * i] = roots[i];
                    roots[2 * i + 1] = roots[i] * e;
                }
                k++;
            }
        }
    }
}

```

```

    }
}

for (int k = 1; k < n; k *= 2) {
    for (int i = 0; i < n; i += 2 * k) {
        for (int j = 0; j < k; j++) {
            Z u = a[i + j];
            Z v = a[i + j + k] * roots[k + j];
            a[i + j] = u + v;
            a[i + j + k] = u - v;
        }
    }
}

}

}

void idft(std::vector<Z> &a) {
    int n = a.size();
    std::reverse(a.begin() + 1, a.end());
    dft(a);
    Z inv = Z(n).inv();
    for (int i = 0; i < n; i++) a[i] *= inv;
}

}

// --- Poly 结构体 (仅保留卷积) ---
struct Poly : public std::vector<Z> {
    using std::vector<Z>::vector; // 使用 vector 的构造函数

    // 乘法重载: 核心功能
    friend Poly operator*(Poly a, Poly b) {
        if (a.empty() || b.empty()) return Poly();
        int sz = a.size() + b.size() - 1;
        int n = 1;
        while (n < sz) n *= 2;
        a.resize(n);
        b.resize(n);
        NTT::dft(a);
        NTT::dft(b);
        for (int i = 0; i < n; i++) a[i] *= b[i];
        NTT::idft(a);
        a.resize(sz);
        return a;
    }

    // 加法重载 (可选, 方便写代码)
    Poly& operator+=(const Poly &b) {
        if (this->size() < b.size()) this->resize(b.size());
        for (int i = 0; i < b.size(); i++) (*this)[i] += b[i];
        return *this;
    }

    friend Poly operator+(Poly a, const Poly &b) { return a += b; }
};

```

8.素性测试 与 因式分解

```
i64 mul(i64 a, i64 b, i64 m) {
    return static_cast<__int128>(a) * b % m;
}

i64 power(i64 a, i64 b, i64 m) {
    i64 res = 1 % m;
    for (; b >= 1, a = mul(a, a, m))
        if (b & 1)
            res = mul(res, a, m);
    return res;
}

bool isprime(i64 n) {
    if (n < 2)
        return false;
    static constexpr int A[] = {2, 3, 5, 7, 11, 13, 17, 19, 23};
    int s = __builtin_ctzll(n - 1);
    i64 d = (n - 1) >> s;
    for (auto a : A) {
        if (a == n)
            return true;
        i64 x = power(a, d, n);
        if (x == 1 || x == n - 1)
            continue;
        bool ok = false;
        for (int i = 0; i < s - 1; ++i) {
            x = mul(x, x, n);
            if (x == n - 1) {
                ok = true;
                break;
            }
        }
        if (!ok)
            return false;
    }
    return true;
}

std::vector<i64> factorize(i64 n) {
    std::vector<i64> p;
    std::function<void(i64)> f = [&](i64 n) {
        if (n <= 10000) {
            for (int i = 2; i * i <= n; ++i)
                for (; n % i == 0; n /= i)
                    p.push_back(i);
            if (n > 1)
                p.push_back(n);
            return;
        }
        if (isprime(n)) {
            p.push_back(n);
            return;
        }
    }
}
```

```

    auto g = [&](i64 x) {
        return (mul(x, x, n) + 1) % n;
    };
    i64 x0 = 2;
    while (true) {
        i64 x = x0;
        i64 y = x0;
        i64 d = 1;
        i64 power = 1, lam = 0;
        i64 v = 1;
        while (d == 1) {
            y = g(y);
            ++lam;
            v = mul(v, std::abs(x - y), n);
            if (lam % 127 == 0) {
                d = std::gcd(v, n);
                v = 1;
            }
            if (power == lam) {
                x = y;
                power *= 2;
                lam = 0;
                d = std::gcd(v, n);
                v = 1;
            }
        }
        if (d != n) {
            f(d);
            f(n / d);
            return;
        }
        ++x0;
    }
};
f(n);
std::sort(p.begin(), p.end());
return p;
}

```

- 如何获得所有因数?

```

auto vec = factorize(n);
vector<PII> pe;
for(int i = 0, j = 0; i < vec.size(); ) {
    int c = 0;
    while(j < vec.size() && vec[j] == vec[i]) {
        c++;
        j++;
    }
    pe.emplace_back(vec[i], c);
    i = j;
}
vector<ll> fac;

```

```

auto dfs = [&](this auto &&self, ll i, ll x) -> void {
    if(i == pe.size()) {
        fac.push_back(x);
        return;
    }
    auto [P, E] = pe[i];
    for(ll e = 0, p = 1; e <= E; e++, p *= P) {
        self(i + 1, x * p);
    }
};
dfs(0, 1);
sort(fac.begin(), fac.end());

```

9.圆方树

```

//1-indexed
struct BlockCutTree {
    int n;
    vector<vector<int>> adj;
    vector<int> dfn, low, stk;
    int cnt, cur;
    vector<pair<int,int>> edges;
    BlockCutTree() {}
    BlockCutTree(int n) {
        init(n);
    }
    void init(int n) {
        this->n = n;
        adj.assign(n, {});
        dfn.assign(n, -1);
        low.resize(n);
        stk.clear();
        cnt = cur = 0;
        edges.clear();
    }
    void addEdge(int u, int v) {
        adj[u].push_back(v);
        adj[v].push_back(u);
    }
    void dfs(int x) {
        stk.push_back(x);
        dfn[x] = low[x] = cur++;
        for(auto y : adj[x]) {
            if(dfn[y] == -1) {
                dfs(y);
                low[x] = min(low[x], low[y]);
                if(low[y] == dfn[x]) {
                    int v;
                    do {
                        v = stk.back();
                        stk.pop_back();
                    } while (v != y);
                    edges.push_back({x, y});
                }
            }
        }
    }
};

```

```

        edges.emplace_back(n + cnt, v);
    } while (v != y);
    edges.emplace_back(x, n + cnt);
    cnt++;
}
} else {
    low[x] = min(low[x], dfn[y]);
}
}
}
pair<int, vector<pair<int, int>>> work() {
    for(int i = 1; i < n; i++) {
        if(dfn[i] == -1) {
            stk.clear();
            dfs(i);
        }
    }
    return {cnt, edges};
}
};

```

使用教程:

```

BlockCutTree t(n + 1);
for(int i = 1; i <= m; i++) {
    int u, v;
    cin >> u >> v;
    t.addEdge(u, v);
}
auto [cnt, edges] = t.work();

vector<vector<int>> tr(n + cnt + 1);
for(auto [u, v] : edges) {
    tr[u].push_back(v);
}

vector<int> fa(n + cnt + 1);
[&](this auto &&self, int x) -> void {
    for(auto y : tr[x]) {
        fa[y] = x;
        self(y);
    }
}(1);

int len = 0;
vector<vector<array<int, 2>>> e(cnt + 1);
for(int x = 1; x <= n; x++) {
    for(int y : t.adj[x]) {
        if(x > y) {
            continue;
        }
        int z;
        if(fa[x] == fa[y] || (fa[x] != 0 && fa[fa[x]] == y)) {

```

```

        z = fa[x];
    } else {
        z = fa[y];
    }
    e[z - n].push_back({x, y});
}
}
vector<int> vis(n + 1);
vector<vector<int>> adj(n + 1);
for(int c = 1; c <= cnt; c++) {
    vector<int> point;
    for(auto [x, y] : e[c]) {
        if(vis[x] != c) {
            vis[x] = c;
            point.push_back(x);
            adj[x].clear();
        }
        if(vis[y] != c) {
            vis[y] = c;
            point.push_back(y);
            adj[y].clear();
        }
        adj[x].push_back(y);
        adj[y].push_back(x);
    }
}
}

```

10.HLD

```

// [in[u], out[u] - 1] 是以u为根的子树的dfs序区间
struct HLD {
    int n;
    vector<int> siz, top, dep, parent, in, out, seq;
    vector<vector<int>> adj;
    int cur;

    HLD() {}
    HLD(int n) {
        init(n);
    }
    void init(int n) {
        this->n = n;
        siz.resize(n);
        top.resize(n);
        dep.resize(n);
        parent.resize(n);
        in.resize(n);
        out.resize(n);
        seq.resize(n);
        cur = 1;
        adj.assign(n, {});
    }
}

```

```

void addEdge(int u, int v) {
    adj[u].push_back(v);
    adj[v].push_back(u);
}

void work(int root = 1) {
    top[root] = root;
    dep[root] = 0;
    parent[root] = 0;
    dfs1(root);
    dfs2(root);
}

void dfs1(int u) {
    if(parent[u]) {
        adj[u].erase(find(adj[u].begin(), adj[u].end(), parent[u]));
    }

    siz[u] = 1;
    for(auto &v : adj[u]) {
        parent[v] = u;
        dep[v] = dep[u] + 1;
        dfs1(v);
        siz[u] += siz[v];
        if(siz[v] > siz[adj[u][0]]) {
            swap(v, adj[u][0]);
        }
    }
}

void dfs2(int u) {
    in[u] = cur++;
    seq[in[u]] = u;
    for(auto v : adj[u]) {
        top[v] = v == adj[u][0] ? top[u] : v;
        dfs2(v);
    }
    out[u] = cur;
}

int lca(int u, int v) {
    while(top[u] != top[v]) {
        if(dep[top[u]] > dep[top[v]]) {
            u = parent[top[u]];
        } else {
            v = parent[top[v]];
        }
    }
    return dep[u] < dep[v] ? u : v;
}

int dist(int u, int v) {
    return dep[u] + dep[v] - 2 * dep[lca(u, v)];
}

int jump(int u, int k) {
    if(dep[u] < k) {
        return 0;
    }
}

```

```

    int d = dep[u] - k;
    while(dep[top[u]] > d) {
        u = parent[top[u]];
    }
    return seq[in[u] - dep[u] + d];
}
bool isAncestor(int u, int v) {
    return in[u] <= in[v] && in[v] < out[u];
}
int rootedParent(int u, int v) {
    swap(u, v);
    if(u == v) {
        return u;
    }
    if(!isAncestor(u, v)) {
        return parent[u];
    }
    auto it = upper_bound(adj[u].begin(), adj[u].end(), v, [&](int x, int y) {
        return in[x] < in[y];
    }) - 1;
    return *it;
}
int rootedSize(int u, int v) {
    if(u == v) {
        return n - 1;
    }
    if(!isAncestor(v, u)) {
        return siz[v];
    }
    return n - 1 - siz[rootedParent(u, v)];
}
int rootedLca(int a, int b, int c) {
    return lca(a, b) ^ lca(b, c) ^ lca(c, a);
}
};

```

11.线段树

```

template<class Info, class Tag>
struct LazySegmentTree {
    int n;
    vector<Info> info;
    vector<Tag> tag;
    LazySegmentTree() : n(0) {}
    LazySegmentTree(int n_, Info v_ = Info()) {
        init(n_, v_);
    }
    template<class T>
    LazySegmentTree(vector<T> init_) {
        init(init_);
    }
}

```

```

void init(int n_, Info v_ = Info()) {
    init(vector(n_, v_));
}
template<class T>
void init(vector<T> init_) {
    n = init_.size();
    info.assign(4 << __lg(n), Info());
    tag.assign(4 << __lg(n), Tag());
    auto build = [&](auto &&self, int o, int l, int r) {
        if(l == r) {
            info[o] = init_[l];
            return;
        }
        int m = l + r >> 1;
        self(self, o << 1, l, m);
        self(self, o << 1 | 1, m + 1, r);
        pull(o);
    };
    build(build, 1, 0, n - 1);
}
void pull(int o) {
    info[o] = info[o << 1] + info[o << 1 | 1];
}
void apply(int o, const Tag &v) {
    info[o].apply(v);
    tag[o].apply(v);
}
void push(int o) {
    apply(o << 1, tag[o]);
    apply(o << 1 | 1, tag[o]);
    tag[o] = Tag();
}
void modify(int o, int l, int r, int x, const Info &v) {
    if(l == r) {
        info[o] = v;
        return ;
    }
    int m = l + r >> 1;
    push(o);
    if(x <= m) modify(o << 1, l, m, x, v);
    else modify(o << 1 | 1, m + 1, r, x, v);
    pull(o);
}
void modify(int x, const Info &v) {
    modify(1, 0, n - 1, x, v);
}
Info rangeQuery(int o, int l, int r, int x, int y) {
    if(l > y || r < x) {
        return Info();
    }
    if(x <= l && r <= y) {
        return info[o];
    }
}

```

```

        int m = l + r >> 1;
        push(o);
        return rangeQuery(o << 1, l, m, x, y) + rangeQuery(o << 1 | 1, m + 1, r, x, y);
    }
Info rangeQuery(int l, int r) {
    return rangeQuery(1, 0, n - 1, l, r);
}
void rangeApply(int o, int l, int r, int x, int y, const Tag &v) {
    if(l > y || r < x) {
        return ;
    }
    if(x <= l && r <= y) {
        apply(o, v);
        return;
    }
    int m = l + r >> 1;
    push(o);
    rangeApply(o << 1, l, m, x, y, v);
    rangeApply(o << 1 | 1, m + 1, r, x, y, v);
    pull(o);
}
void rangeApply(int l, int r, const Tag &v) {
    rangeApply(1, 0, n - 1, l, r, v);
}
template<class F>
int findFirst(int o, int l, int r, int x, int y, F &&pred) {
    if(l > y || r < x) {
        return -1;
    }
    if(x <= l && r <= y && !pred(info[o])) {
        return -1;
    }
    if(l == r) {
        return l;
    }
    int m = l + r >> 1;
    push(o);
    int res = findFirst(o << 1, l, m, x, y, pred);
    if(res == -1) {
        res = findFirst(o << 1 | 1, m + 1, r, x, y, pred);
    }
    return res;
}
template<class F>
int findFirst(int l, int r, F &&pred) {
    return findFirst(1, 0, n - 1, l, r, pred);
}
template<class F>
int findLast(int o, int l, int r, int x, int y, F &&pred) {
    if(l > y || r < x) {
        return -1;
    }
    if(x <= l && r <= y && !pred(info[o])) {

```

```

        return -1;
    }
    if(l == r) {
        return l;
    }
    int m = l + r >> 1;
    push(o);
    int res = findLast(o << 1 | 1, m + 1, r, x, y, pred);
    if(res == -1) {
        res = findLast(o << 1, l, m, x, y, pred);
    }
    return res;
}
template<class F>
int findLast(int l, int r, F &&pred) {
    return findLast(l, 0, n - 1, l, r, pred);
}
};

struct Tag {
    int add = 0;
    void apply(const Tag &t) {
        add += t.add;
    }
};

struct Info {
    int min = inf;
    void apply(const Tag &t) {
        min += t.add;
    }
};

Info operator + (const Info &l, const Info &r) {
    return {min(l.min, r.min)};
}

```

12.一般图最大匹配

在任意一个无向图（不仅仅是二分图）中，寻找一个**边集**，使得这些边中**没有任何两条边共用同一个顶点**，并且让这个边集包含的边数**尽可能多**（即最大匹配）。

```

struct Graph {
    int n;
    std::vector<std::vector<int>>> e;
    Graph(int n) : n(n), e(n) {}
    void addEdge(int u, int v) {
        e[u].push_back(v);
        e[v].push_back(u);
    }
}

```

```

std::vector<int> findMatching(int m, const auto &init) {
    std::vector<int> match(n, -1), vis(n), link(n), f(n), dep(n);
    for (auto [x, y] : init) {
        match[x] = y;
        match[y] = x;
    }
    // disjoint set union
    auto find = [&](int u) {
        while (f[u] != u)
            u = f[u] = f[f[u]];
        return u;
    };
    auto lca = [&](int u, int v) {
        u = find(u);
        v = find(v);
        while (u != v) {
            if (dep[u] < dep[v])
                std::swap(u, v);
            u = find(link[match[u]]);
        }
        return u;
    };
    std::queue<int> que;
    auto blossom = [&](int u, int v, int p) {
        while (find(u) != p) {
            link[u] = v;
            v = match[u];
            if (vis[v] == 0) {
                vis[v] = 1;
                que.push(v);
            }
            f[u] = f[v] = p;
            u = link[v];
        }
    };
    // find an augmenting path starting from u and augment (if exist)
    auto augment = [&](int u) {
        while (!que.empty())
            que.pop();
        std::iota(f.begin(), f.end(), 0);
        // vis = 0 corresponds to inner vertices, vis = 1 corresponds to outer vertices
        std::fill(vis.begin(), vis.end(), -1);
        que.push(u);
        vis[u] = 1;
        dep[u] = 0;
        int y = -1;
        while (!que.empty()) {
            int u = que.front();
            que.pop();
            if (u >= m) {
                y = u;
            }
            for (auto v : e[u]) {

```

```

        if (vis[v] == -1) {
            vis[v] = 0;
            link[v] = u;
            dep[v] = dep[u] + 1;
            // found an augmenting path
            if (match[v] == -1) {
                for (int x = v, y = u, temp; y != -1; x = temp, y = x == -1 ?
-1 : link[x]) {

                    temp = match[y];
                    match[x] = y;
                    match[y] = x;
                }
                return;
            }
            vis[match[v]] = 1;
            dep[match[v]] = dep[u] + 2;
            que.push(match[v]);
        } else if (vis[v] == 1 && find(v) != find(u)) {
            // found a blossom
            int p = lca(u, v);
            blossom(u, v, p);
            blossom(v, u, p);
        }
    }
}

if (y != -1) {
    for (int x = -1, temp; y != -1; x = temp, y = x == -1 ? -1 : link[x]) {
        temp = match[y];
        if (x != -1) {
            match[x] = y;
        }
        match[y] = x;
    }
}

};
// find a maximal matching greedily (decrease constant)
// auto greedy = [&]() {
//     for (int u = 0; u < n; ++u) {
//         if (match[u] != -1)
//             continue;
//         for (auto v : e[u]) {
//             if (match[v] == -1) {
//                 match[u] = v;
//                 match[v] = u;
//                 break;
//             }
//         }
//     }
// };
// greedy();
for (int u = 0; u < m; ++u)
    if (match[u] == -1)
        augment(u);

```

```
        return match;  
    }  
};
```